



Week 13 (Deadlock Management)

✓ Bridge Crossing Example (compare deadlock to real-world)

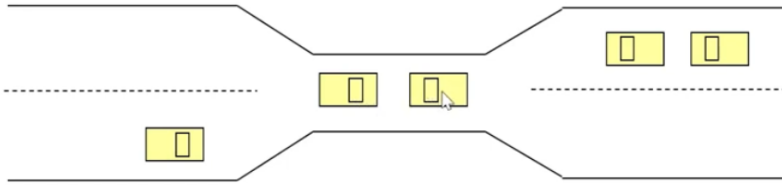
✓ Deadlock and Starvation

- deadlock characteristics
- Modeling deadlocks
- Method for handling deadlock

| Content

✓ Bridge Crossing Example (compare deadlock to real-world)

Bridge Crossing Example



- ❑ Traffic only in one direction
- ❑ Each section of a bridge can be viewed as a resource
- ❑ If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback)
- ❑ Several cars may have to be backed up if a deadlock occurs
- ❑ Starvation is possible
- ❑ Note – Most OSes do not prevent or deal with deadlocks

- if no one willing to go back → deadlock
- the point is, if deadlock occurs → the way to solve the problem is okay, there are cars that need to back up
- and sometimes only 1 car can't solve the problem, more than 1 car need to back up
- Starvation is possible?
 - what if one side agrees to back up, and the other side just continues without stopping

Most OS do not prevent or deal with deadlocks

- the point is OS pretend that there is no deadlock in this world, and you will learn why OS does that, what this means for us.
- if we have to know that, do synchronization

✓ Deadlock and Starvation

Deadlock will occur if 4 conditions occur at the same time

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- **Starvation** – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended.

- in many cases, deadlock is not easy to occur
- but sometimes, writing program this way can cause deadlock
 - for example S and Q are binary semaphore and all of them initialized to 1 (1 marble in a jar)
 - OS may say p0 work first
 - take 1 marble from S jar
 - OS may like p1 next
 - take 1 marble from Q jar
 - come back to p0
 - wait forever (wait Q)

The problem that leads to deadlock → Starvation

- the process may never work

Same thing between deadlock and starvation is that that process never finishes

- for deadlock it always involves more than 1 process → and if we not deal with it (it may extend to many processes) (traffic)

Deadlock characteristics

(at the same time, 4 condition occur → deadlock occur)

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- ❑ **Mutual exclusion:** only one process at a time can use a resource.
- ❑ **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- ❑ **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- ❑ **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .
- ❑ Removal of only one condition can resolve the deadlock

1. Mutual exclusion

- the mutual exclusion will occur when the process use a resource that allow only 1 process at a time
- the resource can only be use by 1 process at a time.

2. Hold and wait

- process holding some resources and waiting for some resources that are hold by someone else.

3. No preemption

- a resource can be taken back if the process is willing to release it itself (OS cannot do anything).

4. Circular wait

- hold and wait must happen it circular order
 - p1 wait for p2, p2 wait for p3, p3 wait for p1
 - deadlock occur bettern p1, p2, p3

If os want to interfere, it jsut have to avoid one condition, it seem easy?

- but it not easy, and OS said no, it will not do this. Not worth it

Modeling Deadlock

Modeling Deadlocks

- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - request
 - use
 - release

- we want to findout if deadlock occur in our system or not, how can we modelling deadlock.
- use graph
 - node
 - $R \rightarrow$ resource
 - $P \rightarrow$ process

Resource-Allocation Graph

A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- request edge – directed edge $P_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$

- transition from process to resource
 - request edge process request resource
- transition from resource to process
 - resource is assign to process

Resource-Allocation Graph (Cont.)

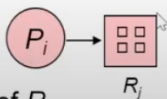
- Process



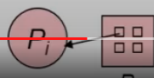
- Resource Type with 4 instances



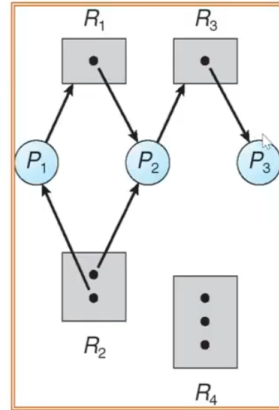
- P_i requests instance of R_j



- P_i is holding an instance of R_j



Example of a Resource Allocation Graph

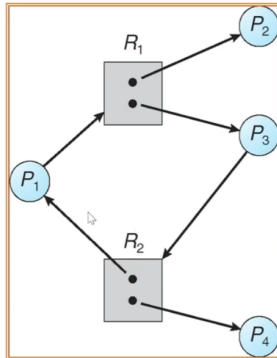


- in this system
 - 3 processes
 - 4 resources
 - $R_1 \rightarrow 1$ item
 - $R_2 \rightarrow 2$ item
 - item $\rightarrow$ number of resources
 - 2 printer
- p_1 is request to r_1 , and hold one of r_2 .

we model the system with resource allocation graph

- how we model?
- we take a look, and see if there is a cycle in the graph
- in this, we have cycle
 - in this system, there is a deadlock

Graph With A Cycle But No Deadlock



- There cycle, but no deadlock
- we take a look at cycle, but it doesn't mean always deadlock.
 - P3 whole 1 of R1, but if P2 release, P1 can continue
 - each resource is mutual exclusion, but we have more than one resource

Conclusion

Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock.
 - If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock.
 - if several instances per resource type, possibility of deadlock.
- no cycle, no circular wait $\rightarrow$ no deadlock for sure
 - one instance per resource type $\rightarrow$ deadlock
 - several instance $\rightarrow$ maybe

Methods for Handling Deadlocks

(Assume that OS has to handle deadlock)

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

- so there are 3 ways that os can handle deadlock (if it was)

- 2 major way

1. Ensure the system will never enter deadlock
2. Allow but recover later.
3. (choose) pretend deadlock doesn't exist.

1. Never allow

- Deadlock prevention

Deadlock Prevention

Restrain the ways request can be made

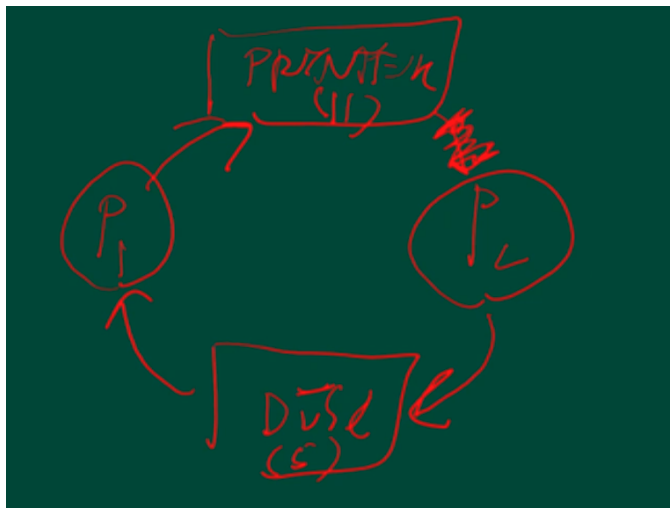
- **Mutual Exclusion** – not required for sharable resources; must hold for nonsharable resources
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none
 - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption** –
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration

- OS need to prevent one of the condition not to occur (out of 4)
 - prevent mutual exclusion: OS can't do, nature of resource. sometimes, can't be interfere by os
 - hold and wait: os must have condition like before you start you hve to get all resource you want first. IF the process dont' get both of them just wait.
 - seems good, why os doesn't do
 - if os do, there might be some process that never get to run.

- No preemption
 - not easy, sometime, you shouldn't get the resource back.
 - contradict with hold and wait?
- Circular wait
 - what if os said that if you not get lower number, you can't hold the higher number
 - P2 have to be remove. (may cause starvation)



- why os doesn't do this? where number come from? maybe stat?
 - P2 is sad
 - condition that os set might not good for all process
- Deadlock avoidance
 - try to avoid the unsafe condition

Deadlock Avoidance

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock.
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.